# Unit 3 — Processor and Control Unit: Exam Revision Sheet

Condensed from the Unit 3 notes and the video script. These are **practice questions built from your notes, not a prediction of your paper.**

---

## How to use this sheet

| Time left | Do this |
|---|---|
| 1 hour | Section 1 (short answers) + Section 3 (formulas) + Section 6 (mistakes) |
| 3 hours | Add Section 2 (long-answer plans) and practise drawing the 5 diagrams in Section 5 |
| 1 day+ | Work all problems in Section 4 without looking at the answers, then check |

**Answer-writing rule:** for every long answer, give **definition → diagram → explanation → example**. A labelled diagram plus a worked example usually carries a lot of weight.

---

## Section 1 — Short answers (2–3 lines each)

1. **Eight great ideas:** Design for Moore's Law; Use abstraction to simplify design; Make the common case fast; Performance via parallelism; Performance via pipelining; Performance via prediction; Hierarchy of memories; Dependability via redundancy.
2. **Moore's Law:** the number of transistors in an integrated circuit doubles approximately every two years (Gordon Moore, co-founder of Intel). Architects must design for where technology will be when the design *finishes*.
3. **ISA:** the functions and capabilities of the CPU — its instruction set, data formats and the programming language it understands. **Microarchitecture** (computer organization): how a particular processor implements the ISA.
4. **CISC vs RISC:** CISC — hundreds of instructions, small register set, fewer lines of code, less memory, slower instructions. RISC — small instruction set, few addressing modes, faster, one clock cycle per instruction. MIPS is RISC.
5. **Five functional units:** Input, Memory, ALU, Output, Control unit.
6. **Primary vs secondary memory:** primary works at electronic speed and holds programs while they execute (RAM, ROM, cache); secondary holds large, infrequently accessed data (magnetic disks/tapes, CD-ROM).
7. **Word length:** number of bits in each word. **RAM:** memory where any location can be reached in a short, fixed time after specifying its address. **Memory access time:** time required to access one word.
8. **Cache:** small, fast RAM coupled with the processor, often on the same chip.
9. **MAR / MDR:** MAR holds the address of the location to be accessed; MDR holds the data to be written into or read out of that location.
10. **Interrupt:** a request signal from an I/O device for service by the processor. The processor runs an **interrupt service routine**, after saving its state, then restores the state and resumes the program.

11. **Response time vs throughput:** response (execution) time = time from start to completion of a task; throughput (bandwidth) = total work done in a given time.
12. **CPU execution time:** actual time the CPU spends computing for a task. **User CPU time:** time in the program itself. **System CPU time:** time in the OS on behalf of the program. CPU time excludes I/O and other programs.
13. **CPI:** average number of clock cycles each instruction takes to execute.
14. **Performance equation:** CPU time = Instruction count × CPI × Clock cycle time = (Instruction count × CPI) / Clock rate.
15. **Factors:** algorithm → IC, possibly CPI; language → IC, CPI; compiler → IC, CPI; ISA → IC, CPI and clock rate.
16. **Instruction / instruction set:** the words of a computer's language are instructions; its vocabulary is the instruction set.
17. **Register operands:** MIPS has a 32 × 32-bit register file, numbered 0–31, used for frequently accessed data; *t*0–t9 temporaries, *s*0–s7 saved variables.
18. **Big vs Little Endian:** Big Endian (MIPS) — most-significant byte at the lowest address; Little Endian — least-significant byte at the lowest address.
19. **No subtract-immediate:** use a negative constant, e.g. `addi $s2, $s1, -1`.
20. **Addressing mode:** the method used to identify the location of an operand. MIPS: immediate, register, base/displacement, PC-relative, pseudodirect.
21. **PC-relative vs pseudodirect:** PC-relative *adds* (PC + 4) + offset×4 (branches, 16-bit offset); pseudodirect *concatenates* PC[31:28] ‖ 26-bit address ‖ 00 (jumps).
22. **Formats:** R = op6 rs5 rt5 rd5 shamt5 funct6; I = op6 rs5 rt5 imm16; J = op6 target26. All 32 bits.
23. **funct field:** extends the opcode; add and sub both have op = 0, funct 32 = add, 34 = sub.
24. **Datapath element:** a unit used to operate on or hold data within a processor (instruction/data memory, register file, ALU, adders).
25. **Register file:** a state element containing a set of registers that can be read and written by supplying a register number; 2 read ports, 1 write port.
26. **Sign extension:** increase the size of a data item by replicating its high-order sign bit into the high-order bits of the larger destination.
27. **Branch target address:** address specified in a branch, which becomes the new PC if taken; = offset field + address of the instruction following the branch.
28. **Branch taken / not taken:** taken — condition true, PC becomes the branch target; not taken — condition false, PC becomes the next sequential instruction.
29. **Delayed branch:** the instruction immediately following the branch is always executed, regardless of the condition.
30. **Separate instruction and data memories:** in a single-cycle design no resource can be used twice per instruction, so elements needed twice are duplicated.
31. **Multiplexor:** a data selector that chooses one of several inputs based on its control lines.
32. **Single-cycle design impractical:** the clock cycle must stretch to fit the longest instruction (lw), so every instruction takes that long.
33. **Pipelining:** increases throughput, does **not** reduce latency of one instruction.
34. **slt vs sltu:** signed vs unsigned comparison; all-ones vs 1 → slt gives 1 (−1 < 1), sltu gives 0 (4,294,967,295 > 1).
35. **jal / jr:** jal puts the return address in $ra and jumps; jr $ra copies $ra to PC.

## Section 2 — Long-answer plans (what to write, in order)

### 2.1 Explain MIPS addressing modes with diagrams

1. Definition of addressing mode.
2. For **each of the 5 modes**: one-line definition (notes' wording) → format box → how the operand/address is formed → example instruction.
3. **Draw** the notes' 5-row diagram (operand shaded).
4. Worked numbers: `lw $t0,32($s3)` with $s3 = 1000 → EA 1032; beq offset 3 at PC 0x00400010 → target 0x00400020; j target.
5. Comparison table (format, calculation, used by).

### 2.2 Explain a basic MIPS implementation

1. The subset: lw, sw | add, sub, AND, OR, slt | beq, j.
2. First two steps identical: fetch using PC; read one or two registers.
3. ALU use: address (lw/sw), operation (R-type), comparison (beq).
4. After ALU: memory access (lw/sw), write-back (R, lw), PC = PC + 4 or branch target.
5. **Draw Figure 1.1**, then explain the two omissions → multiplexors and control.
6. **Draw Figure 1.2** and explain the three muxes (PC source via Branch AND Zero; ALU vs memory to register; register vs offset to ALU).
7. One line: single-cycle, impractical because of the longest instruction.

### 2.3 Building a datapath / explain the datapath elements

1. Define datapath element and PC.
2. Fetch: PC + instruction memory + adder (Figure 2.2) — instruction memory needs no read signal, PC written every cycle.
3. R-type: register file (4 inputs, 2 outputs, 5-bit register numbers, RegWrite) + ALU (4-bit ALU operation, Zero) — Figure 2.3.
4. lw/sw: data memory (MemRead, MemWrite) + sign-extend 16→32 — Figure 2.4.
5. beq: sign-extend → shift left 2 → adder with PC + 4; ALU Zero → branch logic — Figure 2.5. Shift left 2 is only wiring.
6. Combine: muxes ALUSrc, MemtoReg, PCSrc; separate memories; **draw Figure 4.11**.
7. Trace one instruction (lw is best — it uses everything).

### 2.4 Functional units of a computer

Block diagram → input (keyboard → binary code) → memory (primary: words, addresses, word length, RAM, access time, ROM, cache; secondary: examples) → ALU (operands in registers) → output (examples) → control unit (nerve centre, timing signals).

### 2.5 Basic operational concepts

`Add LOCA, R0` and its 3 steps → `Load LOCA, R1; Add R1, R0` → PC, IR, MAR, MDR roles → **draw processor–memory connection diagram** → the 12 operating steps → interrupts and ISR.

### 2.6 Eight great ideas

Each idea: heading + 2 lines from the notes + one example. Draw the memory pyramid for idea 7.

### 2.7 CPU performance

Response time vs throughput → Performance = 1/Execution time → "n times as fast" → CPU time, user/system time → equations → factors table → **one fully solved problem**.

### 2.8 Instruction formats / representing instructions

Machine code, 32-bit words, register numbers → R, I, J diagrams with bit widths → field meanings → encode `add $t0,$s1,$s2` (0x02324020) and `lw $t0,32($s3)` → notes' opcode table (add/sub/addi/lw/sw).

### 2.9 Operations and operands / logical and decision-making instructions

Operand types (register, memory, immediate) with examples → `f = (g+h) – (i+j)` → logical table (sll, srl, and, or, nor) with bit example → beq/bne → compile the if-else and the while loop from the notes → slt/sltu.

---

## Section 3 — Formula sheet

```
Performance = 1 / Execution time
n = Performance_X / Performance_Y = Time_Y / Time_X        (slower time on top)
CPU time = CPU clock cycles × Clock cycle time = CPU clock cycles / Clock rate
CPU clock cycles = IC × CPI = Σ (CPI_i × C_i)
CPU time = IC × CPI × Clock cycle time = IC × CPI / Clock rate
CPI (overall) = total cycles / total instructions
Clock rate = 1 / Clock cycle time      (1 GHz ↔ 1 ns, 2 GHz ↔ 0.5 ns, 4 GHz ↔ 250 ps)

Base address:     EA = Reg[rs] + sign-extend(offset)
Branch target:    (PC + 4) + (sign-extend(offset) << 2)
Jump address:     PC[31:28] ‖ target26 ‖ 00
Array byte offset = index × 4
```

---

## Section 4 — Practice problems (cover the answers first)

**P1.** A program executes $10^9$ instructions, CPI = 1.5, clock = 3 GHz. CPU time?

> Cycles = $10^9 \times 1.5 = 1.5 \times 10^9$. Time = $1.5 \times 10^9 / 3 \times 10^9$ = **0.5 s**.

**P2.** A runs a program in 20 s, B in 25 s. How much faster is A?

> n = 25 / 20 = **1.25** — A is 1.25 times as fast as B.

**P3.** Clock cycle time 0.5 ns, $4 \times 10^8$ instructions, CPI 2.5. Clock rate and CPU time?

> Rate = 1 / 0.5 ns = **2 GHz**. Cycles = $4 \times 10^8 \times 2.5 = 10^9$. Time = $10^9 \times 0.5 \times 10^{-9}$ = **0.5 s**.

**P4.** Same ISA. P1: 3 GHz, CPI 1.5. P2: 2.5 GHz, CPI 1.0. P3: 4 GHz, CPI 2.2. Which has the highest performance (instructions per second)?

> IPS = clock rate / CPI. P1 = $3\times10^9/1.5 = 2.0\times10^9$; P2 = $2.5\times10^9/1.0 = 2.5\times10^9$; P3 = $4\times10^9/2.2 \approx 1.82\times10^9$. **P2 is fastest**, even with the lowest clock rate.

**P5.** A program takes 12 s on computer A with a 3 GHz clock. Computer B will need 1.5 × as many cycles. What clock rate does B need to run it in 9 s?

> Cycles_A = $12 \times 3 \times 10^9 = 36 \times 10^9$. Cycles_B = $1.5 \times 36 \times 10^9 = 54 \times 10^9$. Rate_B = $54 \times 10^9 / 9$ = **6 GHz**.

**P6.** CPI: class A = 1, B = 2, C = 3. A code sequence has 3 A, 2 B, 1 C. Cycles and CPI?

> IC = 6. Cycles = 3×1 + 2×2 + 1×3 = 10. CPI = 10/6 = **1.67**.

**P7.** `beq` at 0x00400020 with offset field 5. Target?

> PC + 4 = 0x00400024; 5 << 2 = 20 = 0x14; target = **0x00400038**.

**P8.** `bne` at 0x00400100 with offset field −3. Target?

> PC + 4 = 0x00400104; −3 << 2 = −12 = −0xC; target = **0x004000F8**.

**P9.** `j` at PC 0x00400040 with target field 0x0100100. New PC?

> 0x0100100 << 2 = 0x0400400; PC[31:28] = 0000; new PC = **0x00400400**.

**P10.** $s2 = 0x10008000. Effective address of `lw $t1, -8($s2)`?

> 0x10008000 − 8 = **0x10007FF8**.

**P11.** Encode `sub $t2, $s3, $s4`.

> op 0 | rs 19 | rt 20 | rd 10 | shamt 0 | funct 34 000000 10011 10100 01010 00000 100010 = **0x02745022**.

**P12.** Encode `lw $t1, 8($s0)`.

> op 35 | rs 16 | rt 9 | imm 8 → 100011 10000 01001 0000000000001000 = **0x8E090008**.

**P13.** Encode `addi $t0, $t0, -4`.

> op 8 | rs 8 | rt 8 | imm −4 = 0xFFFC → 001000 01000 01000 1111111111111100 = **0x2108FFFC**.

**P14.** Compile `A[10] = A[2] + h`; with h in $s2, base of A in $s3.

> `lw $t0, 8($s3)` · `add $t0, $t0, $s2` · `sw $t0, 40($s3)` (offsets 2×4, 10×4).

**P15.** $t1 = 0x00003C00, $t2 = 0x00000DC0. Find `and`, `or`, and `nor $t0,$t1,$zero`.

> AND = **0x00000C00**; OR = **0x00003DC0**; NOT $t1 = **0xFFFFC3FF**.

---

## Section 5 — Diagrams you must be able to draw from memory

1. **Five functional units** block diagram (Input/Output, Memory, ALU + Control).
2. **Memory hierarchy pyramid** (registers → cache → RAM → storage).
3. **R / I / J formats** with field names and bit widths.
4. **Five addressing modes** (notes' page-12 figure, operand shaded).
5. **Complete single-cycle datapath** (Figure 4.11): PC, instruction memory, PC + 4 adder, register file, sign-extend, shift left 2, branch adder, ALUSrc mux, ALU (Zero), data memory, MemtoReg mux, PCSrc mux, with control labels RegWrite, ALUSrc, ALU operation, MemRead, MemWrite, MemtoReg, PCSrc.

Practice tip: draw #5 three times — once fast, once with labels, once tracing lw in colour.

---

## Section 6 — Common mistakes to avoid

- Branch target uses **PC + 4**, not PC, and the offset is **×4** (shift left 2).
- Pseudodirect is **concatenation**, not addition.
- Array offset is **index × 4**, not the index (A[8] → 32).
- In `lw`, **rt is the destination**; in `sw`, rt is the value stored.
- "n times as fast": put the **slower** time on top.
- Convert units first: GHz → $\times 10^{9}$ Hz, ns → $\times 10^{-9}$ s, ps → $\times 10^{-12}$ s.
- Don't judge speed by CPI or clock rate alone — compute CPU time (P4, notes' Problem 3).
- Pipelining raises **throughput**, not single-instruction latency.
- Register number fields are **5 bits** ($2^5 = 32$); immediate is **16 bits**; jump target **26 bits**.
- Instruction memory needs no read signal; data memory needs MemRead/MemWrite.
- The constant range is $-2^{15}$ to $2^{15} - 1$ (the notes print it as "−215 to +215 − 1").